// Day 1 of 10

Day 1

10 Spring Boot Mistakes

Developers make these every day.
Don't be one of them.

→ Swipe through all 10

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #01

Field Injection @Autowired

Makes testing hard & hides dependencies

```
// Bad
@Autowired
private UserService userService;

// Good
private final UserService userService;
public UserController(UserService svc){
  this.userService = svc;
}
```

💡 Constructor injection = testable & immutable

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #02

Returning Entities Directly

Exposes DB structure & risks data leaks

```
// Bad
public User getUser(Long id){
  return userRepo.findById(id).get();
}

// Good
public UserDTO getUser(Long id){
  User u = userRepo.findById(id).get();
  return new UserDTO(u.getName(),u.getEmail());
}
```

💡 Always use DTOs for API responses

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #03

No Exception Handling

Unhandled errors crash your API

```
// Bad
public User getUser(Long id){
  return repo.findById(id).get(); // throws!
}

// Good
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity handle(Exception e){
  return ResponseEntity.status(404)
    .body(e.getMessage());
}
```

💡 Use @ControllerAdvice globally

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #04

N+1 Query Problem

Kills performance with hundreds of queries

```
// Bad
orders.forEach(o → {
  o.getUser().getName(); // 1 query per row!
});

// Good
@Query("SELECT o FROM Order o
  JOIN FETCH o.user")
List findAllWithUser();
```

💡 Always check SQL logs in development

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #05

Hardcoding Config Values

Makes app impossible to configure safely

```
// Bad
String key = "supersecret123";
String url = "http://localhost/db";

// Good
@Value("${jwt.secret}")
private String jwtSecret;
// Set via environment variable
```

💡 Never commit secrets to Git

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #06

Business Logic in Controller

Breaks separation of concerns

```
// Bad
@PostMapping("/order")
public Order place(@RequestBody Order o){
  o.setTotal(o.getQty() * o.getPrice());
  return repo.save(o); // wrong layer!
}

// Good
@PostMapping("/order")
public Order place(@RequestBody OrderDTO dto){
  return orderService.placeOrder(dto);
}
```

💡 Controller → Service → Repository always

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #07

No Logging Strategy

Debugging becomes a nightmare

```
// Bad
System.out.println("User: " + user);

// Good
private static final Logger log =
  LoggerFactory.getLogger(UserService.class);
log.info("User created: {}", user.getId());
```

💡 Use SLF4J — never System.out.println

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #08

Skipping Input Validation

Opens API to bad data & crashes

```
// Bad
public User create(@RequestBody User u){
  return repo.save(u); // no validation!
}

// Good
public class UserDTO {
  @NotBlank @Email
  private String email;
  @NotNull @Min(1)
  private Integer age;
}
```

💡 Add @Valid on controller params always

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #09

No Pagination

Loading all records = server crash

```
// Bad
repo.findAll(); // loads ALL records!

// Good
repo.findAll(PageRequest.of(0, 20));
// Returns 20 records at a time
```

💡 Default page size should be 20 or less

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #10

No API Versioning

Breaking changes break your clients

```
// Bad
@GetMapping("/users")
// No version — any change breaks clients

// Good
@GetMapping("/api/v1/users")
// Old clients still work
// New clients use /api/v2/users
```

💡 Always version your API from day 1

Shubham Bhati
linkedin.com/in/bhatishubham

```
done = true
```

Follow for more tips!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham